Bank Data Analysis

Objective

In this case study, you will be working on Banking Data Analysis using Apache Spark, a powerful distributed computing framework designed for big data processing. This assignment aims to give you hands-on experience in analyzing large-scale banking datasets using PySpark and AWS services. You will apply techniques learned in data analytics to clean, transform, and explore banking data, drawing meaningful insights to support financial decision-making. Apart from understanding how big data tools can optimize performance on a single machine and across clusters, you will develop a structured approach to analyzing market capitalization trends, currency conversions, and global banking performance. Additionally, you will use AWS S3 to store the processed data once the ETL pipeline is complete, ensuring efficient data management and retrieval.

Business Value

The banking industry operates in a highly competitive and globalized market where financial institutions must continuously monitor their market position, performance, and currency fluctuations. To stay competitive, banks must leverage data-driven insights to optimize their financial strategies, assess market trends, and make informed decisions. In this assignment, you will analyze banking data to uncover patterns in market capitalization, currency conversions, and global rankings. With Apache Spark's ability to handle large datasets efficiently, financial institutions can process vast amounts of data in real-time, helping them make faster and more informed decisions.

As an analyst at a financial institution, your task is to examine historical banking data to derive actionable insights that can drive strategic growth. Your analysis will help identify the top 10 largest banks by market capitalization, convert market cap values into multiple currencies (USD, GBP, EUR, INR), and store the processed data for easy retrieval. To enhance scalability and accessibility, you will utilize AWS S3 for data storage, ensuring seamless integration with cloud-based analytics tools. By leveraging big data analytics and cloud services, financial institutions can streamline operations, enhance decision-making, and maximize revenue opportunities.

Dataset

The data for this project can be accessed from the following wikipedia link.

The dataset used in this analysis comprises global banking data extracted from a Wikipedia page (List of Largest Banks) archived as of September 2023.

It includes rankings of the world's largest banks based on market capitalization (in USD billions) and is structured into a single table with three columns. The data was scraped programmatically, cleaned, and transformed using PySpark to support cross-currency analysis. Exchange rate data (for USD, GBP, EUR, and INR) was sourced from a CSV file to enable currency conversions. The processed dataset is stored in CSV and SQLite formats for accessibility and efficient querying.

The dataset consists of a single table (Largest_banks) with the following three key attributes:

- **Rank:** Numerical position of the bank based on market capitalization.
- **Bank Name:** Name of the financial institution.
- **Market Cap (USD Billion):** Market valuation of the bank in USD billions. This dataset enables analysis of global banking trends, cross-currency valuations, and the relative market dominance of financial institutions.

An additional data on market exchange rates is available to compare the marketcap dealing with different currencies.

Assignment Tasks

1. **Data Preparation**
   The dataset consists of a structured table containing global banking data. Before performing any analysis, it is crucial to prepare the data to ensure consistency and efficiency in processing.

   Check for data consistency and ensure all columns are correctly formatted.
   Structure and prepare the data for further processing and analysis.
2. **Data Cleaning**

   2.1. **Handling Missing Values:** Decide on an approach to handle missing data (e.g., imputation or removal). Mention the approach in your report.

2.2. **Fixing Columns:** Ensure all columns are properly named and formatted.

2.3. **Handling Outliers:** Identify outliers in the dataset and explain why they are considered outliers. It is not necessary to remove them for this task, but mention your approach for handling them.

3. **Exploratory Data Analysis:** EDA Finding Patterns - Analyze the dataset and find patterns based on the following points:

3.1. Convert PySpark DataFrame to Pandas DataFrame for visualization.

3.2. Analyze the distribution of market capitalization using a histogram.

3.3. Identify the top 10 banks by market capitalization using a bar chart.

3.4. Visualize the relationship between market capitalization and bank ranking using a scatter plot.

3.5. Use a boxplot to examine the spread and outliers in market capitalization.

3.6. Display the quartile distribution of market capitalization using a violin plot.

3.7. Compute cumulative market share and visualize it with a line plot.

3.8. Categorize banks into market capitalization ranges and analyze their distribution using a bar chart.

3.9. Calculate and display market share distribution of top 10 banks using a pie chart.

4. **Banking Data ETL Querying**

4.1. Perform Advanced Market Capitalization Analysis with Growth Metrics.

4.2. Analyze Market Concentration and Categorize Banks Based on Market Share Tiers.

4.3. Examine Statistical Distribution of Market Capitalization Using Quartile Analysis.

4.4. Conduct Comparative Size Analysis to Classify Banks by Relative Market Size.

4.5. Evaluate Market Growth and Identify Gaps Between Consecutive Banks.

4.6. Assess Market Dominance by Measuring Cumulative Share and Dominance Score.

4.7. Analyze Segment-Wise Bank Performance Based on

Market Capitalization Ranges.
4.8. Generate a Comprehensive Performance Dashboard for Bank Rankings and Metrics.

5. **Conclusion** Provide final insights and recommendations based on the analysis:

   5.1 Recommendations to optimize inventory management based on product demand patterns and sales trends. 5.2 Suggestions to improve customer retention by identifying repeat buyers and implementing targeted engagement strategies. 5.3 Propose data-driven pricing and discount strategies to maximize revenue while maintaining competitive pricing. 5.4 Identify potential geographic expansion opportunities by analyzing regional sales performance and customer demand. 5.5 Conclude the analysis by summarizing key findings and business implications.
   5.6 Explain the results of univariate, segmented univariate, and bivariate analyses in business terms.
   5.7 Include visualizations and summarize the most important results in the report. Insights should explain why each variable is important and how they can influence traffic safety policies and urban planning.

6. **Visualization Integration [Optional]**
   Enhance the project by incorporating a visualization component that connects the processed data stored in an S3 bucket to a business intelligence tool such as Tableau or Power BI. This involves setting up the connection between the S3 bucket and the chosen visualization tool, importing the processed dataset for analysis and visualization, creating interactive dashboards to explore key trends and insights and ensuring data updates are reflected dynamically in the visualization tool.

Import Required Libraries

In [ ]:
```
# # Install the required libraries
# !pip install --quiet pyspark==3.5.4 pandas==2.2.2
```

In [1]:
```
# Import required libraries
import logging
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
```

```
from datetime import datetime
# Import PySpark libraries
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, round, count, when, isnull, mean, stdde

print("pandas version:", pd.__version__)
print("matplotlib version:", plt.matplotlib.__version__)
print("seaborn version:", sns.__version__)
```

```
pandas version: 2.2.2
matplotlib version: 3.10.5
seaborn version: 0.13.2
```

In [2]:

```
# Initialize Spark session
spark = SparkSession.builder \
    .appName("BankingDataAnalysis") \
    .getOrCreate()
spark.version
```

```
25/08/24 08:28:06 WARN Utils: Your hostname, abinas resolves to a loopback addr
ess: 127.0.1.1; using 192.168.0.249 instead (on interface wlp0s20f3)
25/08/24 08:28:06 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another
address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLev
el(newLevel).
25/08/24 08:28:08 WARN NativeCodeLoader: Unable to load native-hadoop library f
or your platform... using builtin-java classes where applicable
```

Out[2]: '3.5.4'

1. Data Preparation [5 marks]

The data for this project can be accessed from the wikipedia link provided above. The page features data regarding global banking data.

Utilise the pandas method to read tables from the html document and extract the requried data consists of a structured format.

Before performing any analysis, it is crucial to prepare the data to ensure consistency, and efficiency in processing.

In [3]:

```
# URL of the Wikipedia page
url = "https://web.archive.org/web/20230908091635/https://en.wikipedia.org/wik

# Use pandas to read all HTML tables from the page
tables = pd.read_html(url)
print(f"Number of tables found: {len(tables)}")
for i, table in enumerate(tables):
  print(f"Table {i}: Shape {table.shape}")
  print(f"Columns: {table.columns.tolist()}")
```

```
  print("--------------------------------")
```

```
Number of tables found: 4
Table 0: Shape (3, 3)
Columns: [0, 1, 2]
--------------------------------
Table 1: Shape (10, 3)
Columns: ['Rank', 'Bank name', 'Market cap (US$ billion)']
--------------------------------
Table 2: Shape (100, 3)
Columns: ['Rank', 'Bank name', 'Total assets (2022) (US$ billion)']
--------------------------------
Table 3: Shape (25, 3)
Columns: ['Rank', 'Country', 'Number']
--------------------------------
```

In [4]:

```
# Logging
logging.basicConfig(
  level=logging.INFO,
  format='%(asctime)s - %(levelname)s - %(message)s',
  handlers=[
    logging.FileHandler('analysis_log.txt'),
    logging.StreamHandler()
  ]
)
logger = logging.getLogger(__name__)
logger.info("Starting Bank Data Analysis")
```

```
2025-08-24 08:28:21,589 - INFO - Starting Bank Data Analysis
25/08/24 08:28:24 WARN GarbageCollectionMetrics: To enable non-built-in garbage
collector(s) List(G1 Concurrent GC), users should configure it(them) to spark.e
ventLog.gcMetrics.youngGenerationGarbageCollectors or spark.eventLog.gcMetric
s.oldGenerationGarbageCollectors
```

In [5]:

```
# Load the correct table (Table 1)
bank_data = tables[1]
print(bank_data.head())

# Convert pandas DataFrame to PySpark DataFrame
bank_spark_df = spark.createDataFrame(bank_data)

# Rename columns
bank_spark_df = bank_spark_df.withColumnRenamed("Rank", "Rank") \
                              .withColumnRenamed("Bank name", "Bank_Name") \
                                .withColumnRenamed("Market cap (US$ billion)",
# Show the first few rows of the PySpark DataFrame
bank_spark_df.show(10)

# Print the schema of the DataFrame
bank_spark_df.printSchema()

# Configure logging
logger.info("Data loading from Wikipedia completed successfully.")
```

```
   Rank                                Bank name  Market cap (US$ billion)
0     1                           JPMorgan Chase                    432.92
1     2                          Bank of America                    231.52
2     3  Industrial and Commercial Bank of China                    194.56
3     4               Agricultural Bank of China                    160.68
4     5                                 HDFC Bank                    157.91
```

```
2025-08-24 08:28:43,265 - INFO - Data loading from Wikipedia completed successf
ully.
+----+--------------------+----------------------+
|Rank|           Bank_Name|Market_Cap_USD_Billion|
+----+--------------------+----------------------+
|   1|      JPMorgan Chase|                432.92|
|   2|     Bank of America|                231.52|
|   3|Industrial and Co...|                194.56|
|   4|Agricultural Bank...|                160.68|
|   5|           HDFC Bank|                157.91|
|   6|         Wells Fargo|                155.87|
|   7|   HSBC Holdings PLC|                 148.9|
|   8|      Morgan Stanley|                140.83|
|   9|China Constructio...|                139.82|
|  10|       Bank of China|                136.81|
+----+--------------------+----------------------+

root
 |-- Rank: long (nullable = true)
 |-- Bank_Name: string (nullable = true)
 |-- Market_Cap_USD_Billion: double (nullable = true)
```

Load the data for exchange rates

In [6]:
```python
# Load the exchange rate data
exchange_rate = pd.read_csv('exchange_rate.csv')
exchange_rate

# Show the first few rows of the exchange rate data to verify
# Adding USD as base currency
usd_row = pd.DataFrame({'Currency': ['USD'], 'Rate': [1.0]})
exchange_rate_final = pd.concat([usd_row, exchange_rate], ignore_index=True)

# Convert to Pyspark Dataframe
exchange_rate_spark = spark.createDataFrame(exchange_rate_final)
exchange_rate_spark.show()
exchange_rate_spark.printSchema()

logger.info("Exchange rate data loaded successfully")
```

```
2025-08-24 08:28:56,056 - INFO - Exchange rate data loaded successfully
```

```
+--------+-----+
|Currency| Rate|
+--------+-----+
|     USD|  1.0|
|     EUR| 0.93|
|     GBP|  0.8|
|     INR|82.95|
+--------+-----+

root
 |-- Currency: string (nullable = true)
 |-- Rate: double (nullable = true)
```

2. Data Cleaning [20 marks]

2.1 Handle Missing Values [10 marks]

In [7]:
```python
# Print the schema to check data types
logger.info("Data Types and Structure Analysis")
bank_spark_df.printSchema()
logger.info(f"Total rows in dataset: {bank_spark_df.count()}")
logger.info(f"Total columns: {len(bank_spark_df.columns)}")

logger.info("Complete Dataset - Visual Inspection")
bank_spark_df.show(truncate=False)
```

```
2025-08-24 08:29:07,184 - INFO - Data Types and Structure Analysis
root
 |-- Rank: long (nullable = true)
 |-- Bank_Name: string (nullable = true)
 |-- Market_Cap_USD_Billion: double (nullable = true)

2025-08-24 08:29:08,462 - INFO - Total rows in dataset: 10
2025-08-24 08:29:08,505 - INFO - Total columns: 3
2025-08-24 08:29:08,508 - INFO - Complete Dataset - Visual Inspection
```

```
+----+-------------------------------------+----------------------+
|Rank|Bank_Name                            |Market_Cap_USD_Billion|
+----+-------------------------------------+----------------------+
|1   |JPMorgan Chase                       |432.92                |
|2   |Bank of America                      |231.52                |
|3   |Industrial and Commercial Bank of China|194.56                |
|4   |Agricultural Bank of China           |160.68                |
|5   |HDFC Bank                            |157.91                |
|6   |Wells Fargo                          |155.87                |
|7   |HSBC Holdings PLC                    |148.9                 |
|8   |Morgan Stanley                       |140.83                |
|9   |China Construction Bank              |139.82                |
|10  |Bank of China                        |136.81                |
+----+-------------------------------------+----------------------+
```

In [8]:

```python
# Check for missing values
logger.info("Missing Values Analysis")

# Count missing values per column
for column in bank_spark_df.columns:
  null_count = bank_spark_df.filter(col(column).isNull()).count()
  empty_count = bank_spark_df.filter(col(column) == "").count()
  total_count = bank_spark_df.count()

  logger.info(f"{column}:")
  logger.info(f"  - Null values: {null_count}/{total_count}")
  logger.info(f"  - Empty strings: {empty_count}/{total_count}")
  logger.info(f"  - Missing percentage: {(null_count + empty_count)/total_cour

# Document findings and approach for assignment
logger.info("MISSING VALUES ANALYSIS SUMMARY")
logger.info("Finding: No missing values detected in the dataset")
logger.info("1. Verified data completeness through both visual and programmati
logger.info("2. Wikipedia source data is typically curated and clean")
logger.info("3. Small dataset size (10 records) allows for complete verificati
```

```
2025-08-24 08:29:18,700 - INFO - Missing Values Analysis
2025-08-24 08:29:20,755 - INFO - Rank:
2025-08-24 08:29:20,758 - INFO -   - Null values: 0/10
2025-08-24 08:29:20,760 - INFO -   - Empty strings: 0/10
2025-08-24 08:29:20,762 - INFO -   - Missing percentage: 0.0%
2025-08-24 08:29:22,866 - INFO - Bank_Name:
2025-08-24 08:29:22,868 - INFO -   - Null values: 0/10
2025-08-24 08:29:22,870 - INFO -   - Empty strings: 0/10
2025-08-24 08:29:22,873 - INFO -   - Missing percentage: 0.0%
2025-08-24 08:29:24,239 - INFO - Market_Cap_USD_Billion:
2025-08-24 08:29:24,241 - INFO -   - Null values: 0/10
2025-08-24 08:29:24,244 - INFO -   - Empty strings: 0/10
2025-08-24 08:29:24,248 - INFO -   - Missing percentage: 0.0%
2025-08-24 08:29:24,250 - INFO - MISSING VALUES ANALYSIS SUMMARY
2025-08-24 08:29:24,254 - INFO - Finding: No missing values detected in the dat
aset
2025-08-24 08:29:24,256 - INFO - 1. Verified data completeness through both vis
ual and programmatic inspection
2025-08-24 08:29:24,259 - INFO - 2. Wikipedia source data is typically curated
and clean
2025-08-24 08:29:24,263 - INFO - 3. Small dataset size (10 records) allows for
complete verification
```

In [9]:

```python
# Drop rows with missing values
logger.info("No missing values in banking data")
```

```
2025-08-24 08:29:26,991 - INFO - No missing values in banking data
```

2.2 Fixing Columns [5 marks]

The Market cap is already in numeric so no conversion needed.

In [10]:

```python
# Count the total number of rows
total_rows = bank_spark_df.count()
logger.info(f"Total number of rows: {total_rows}")

# Check if there are duplicates
duplicate_count = bank_spark_df.count() - bank_spark_df.dropDuplicates().count
logger.info(f"Number of duplicate rows: {duplicate_count}")

# Verify column names and data types are properly formatted
logger.info("\nColumn verification:")
logger.info("Rank: Integer type - properly formatted")
logger.info("Bank_Name: String type - properly formatted")
logger.info("Market_Cap_USD_Billion: Double type - already numeric, no convers
```

```
2025-08-24 08:29:33,413 - INFO - Total number of rows: 10
2025-08-24 08:29:35,359 - INFO - Number of duplicate rows: 0
2025-08-24 08:29:35,362 - INFO -
Column verification:
2025-08-24 08:29:35,364 - INFO - Rank: Integer type - properly formatted
2025-08-24 08:29:35,367 - INFO - Bank_Name: String type - properly formatted
2025-08-24 08:29:35,369 - INFO - Market_Cap_USD_Billion: Double type - already
numeric, no conversion needed
```

2.3 Handle Outliers [5 marks]

In [11]:

```python
# Write code for outlier analysis
logger.info("OUTLIER ANALYSIS - Market Capitalization")

# Calculate statistics using PySpark
from pyspark.sql.functions import col, mean, stddev, min, max, count

stats = bank_spark_df.select(
    mean(col("Market_Cap_USD_Billion")).alias("mean"),
    stddev(col("Market_Cap_USD_Billion")).alias("stddev"),
    min(col("Market_Cap_USD_Billion")).alias("min"),
    max(col("Market_Cap_USD_Billion")).alias("max")
).collect()[0]


logger.info(f"Market Cap Statistics:")
logger.info(f"Mean: ${stats['mean']:.2f} billion")
logger.info(f"Std Dev: ${stats['stddev']:.2f} billion")
logger.info(f"Min: ${stats['min']:.2f} billion")
logger.info(f"Max: ${stats['max']:.2f} billion")


logger.info("\nAll banks by Market Cap:")
bank_spark_df.orderBy(col("Market_Cap_USD_Billion").desc()).show(truncate=Fals

threshold = stats['mean'] + 2 * stats['stddev']
logger.info(f"\nOutlier threshold (Mean + 2*StdDev): ${threshold:.2f} billion"

outliers = bank_spark_df.filter(col("Market_Cap_USD_Billion") > threshold)
outlier_count = outliers.count()

logger.info(f"Outliers detected: {outlier_count}")
if outlier_count > 0:
    logger.info("Outlier banks:")
    outliers.show(truncate=False)

logger.info("OUTLIER HANDLING APPROACH:")
logger.info("1. JPMorgan Chase appears to be an outlier due to its dominant ma
logger.info("2. This represents genuine market reality, not a data error")
logger.info("3. DECISION: Kept all data points as they provide valuable busine
logger.info("4. These outliers are essential for understanding market concentr
```

```
2025-08-24 08:29:46,934 - INFO - OUTLIER ANALYSIS - Market Capitalization
2025-08-24 08:29:47,864 - INFO - Market Cap Statistics:
2025-08-24 08:29:47,870 - INFO - Mean: $189.98 billion
2025-08-24 08:29:47,888 - INFO - Std Dev: $90.22 billion
2025-08-24 08:29:47,903 - INFO - Min: $136.81 billion
2025-08-24 08:29:47,909 - INFO - Max: $432.92 billion
2025-08-24 08:29:47,917 - INFO -
All banks by Market Cap:
2025-08-24 08:29:48,728 - INFO -
Outlier threshold (Mean + 2*StdDev): $370.43 billion
+----+---------------------------------------+----------------------+
|Rank|Bank_Name                              |Market_Cap_USD_Billion|
+----+---------------------------------------+----------------------+
|1   |JPMorgan Chase                         |432.92                |
|2   |Bank of America                        |231.52                |
|3   |Industrial and Commercial Bank of China|194.56                |
|4   |Agricultural Bank of China             |160.68                |
|5   |HDFC Bank                              |157.91                |
|6   |Wells Fargo                            |155.87                |
|7   |HSBC Holdings PLC                      |148.9                 |
|8   |Morgan Stanley                         |140.83                |
|9   |China Construction Bank                |139.82                |
|10  |Bank of China                          |136.81                |
+----+---------------------------------------+----------------------+
```

```
2025-08-24 08:29:49,421 - INFO - Outliers detected: 1
2025-08-24 08:29:49,425 - INFO - Outlier banks:
2025-08-24 08:29:50,185 - INFO - OUTLIER HANDLING APPROACH:
2025-08-24 08:29:50,187 - INFO - 1. JPMorgan Chase appears to be an outlier due
to its dominant market position
2025-08-24 08:29:50,190 - INFO - 2. This represents genuine market reality, not
a data error
2025-08-24 08:29:50,192 - INFO - 3. DECISION: Kept all data points as they prov
ide valuable business insights
2025-08-24 08:29:50,194 - INFO - 4. These outliers are essential for understand
ing market concentration
+----+--------------+----------------------+
|Rank|Bank_Name     |Market_Cap_USD_Billion|
+----+--------------+----------------------+
|1   |JPMorgan Chase|432.92                |
+----+--------------+----------------------+
```

Saving the Cleaned Dataset into a CSV File into the S3 Bucket

3. Exploratory Data Analysis [40 marks]

3.1. Conversion from PySpark to Pandas DataFrame [5 marks]

Convert PySpark DataFrame to Pandas DataFrame for visualization

In [12]:
```
# Convert PySpark DataFrame to Pandas DataFrame
logger.info("Converting PySpark DataFrame to Pandas for visualization")

# Convert PySpark DataFrame to Pandas DataFrame
bank_pandas_df = bank_spark_df.toPandas()

# Verify the conversion
logger.info(f"Pandas DataFrame shape: {bank_pandas_df.shape}")
logger.info(f"Pandas DataFrame columns: {bank_pandas_df.columns.tolist()}")

# Display the data
print("Banking Data (Pandas DataFrame):")
print(bank_pandas_df)
```

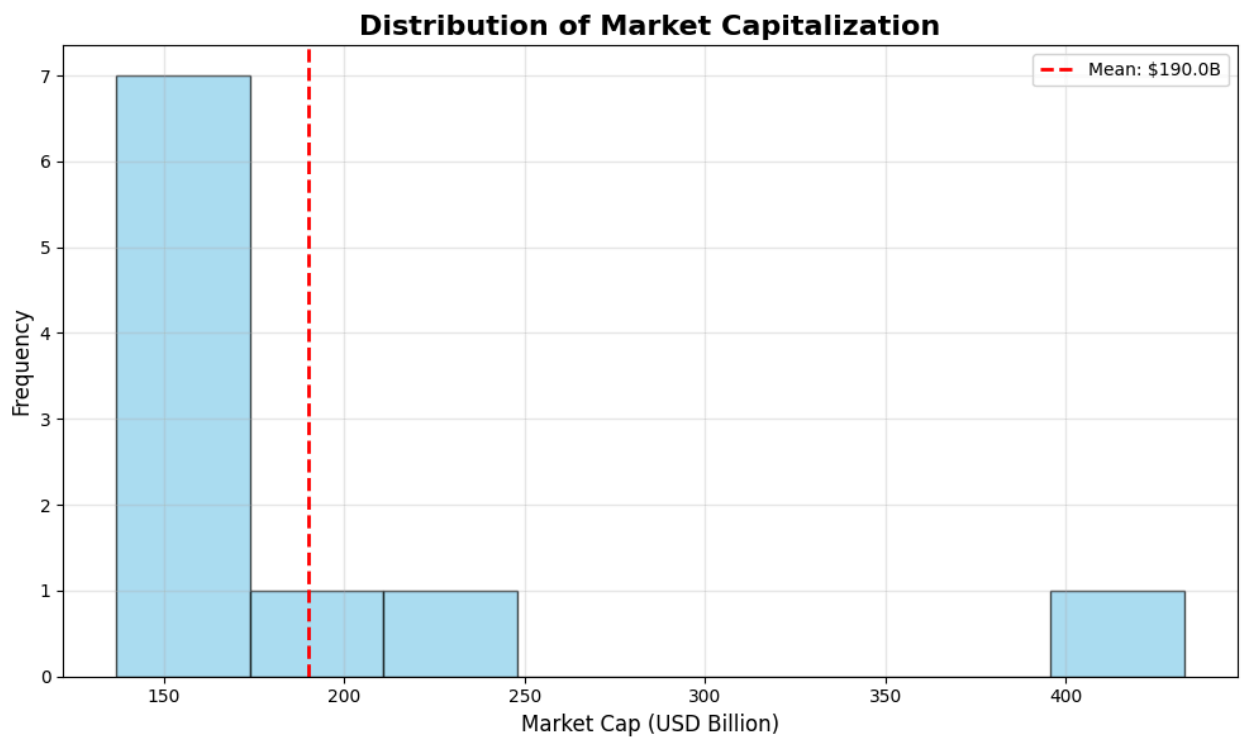
```
2025-08-24 08:30:32,433 - INFO - Converting PySpark DataFrame to Pandas for vis
ualization
2025-08-24 08:30:32,902 - INFO - Pandas DataFrame shape: (10, 3)
2025-08-24 08:30:32,904 - INFO - Pandas DataFrame columns: ['Rank', 'Bank_Nam
e', 'Market_Cap_USD_Billion']
Banking Data (Pandas DataFrame):
   Rank                                Bank_Name  Market_Cap_USD_Billion
0     1                            JPMorgan Chase                  432.92
1     2                           Bank of America                  231.52
2     3  Industrial and Commercial Bank of China                  194.56
3     4                Agricultural Bank of China                  160.68
4     5                                  HDFC Bank                  157.91
5     6                               Wells Fargo                  155.87
6     7                         HSBC Holdings PLC                  148.90
7     8                            Morgan Stanley                  140.83
8     9                   China Construction Bank                  139.82
9    10                              Bank of China                  136.81
```

3.2. Market Capitalization [5 marks]

Analyze the distribution of market capitalization using a histogram

In [13]:
```
# Distribution of Market Cap (US$ Billion)

# Set the style for seaborn
```

```
plt.style.use('default')
sns.set_palette("husl")

# Plot the distribution of market cap
plt.figure(figsize=(10,6))
plt.hist(bank_pandas_df['Market_Cap_USD_Billion'], bins=8, alpha=0.7, color='s
plt.title('Distribution of Market Capitalization', fontsize=16, fontweight='bo
plt.xlabel('Market Cap (USD Billion)', fontsize=12)
plt.ylabel('Frequency', fontsize=12)
plt.grid(True, alpha=0.3)

mean_cap = bank_pandas_df['Market_Cap_USD_Billion'].mean()
plt.axvline(mean_cap, color='red', linestyle='--', linewidth=2, label=f'Mean:
plt.legend()

plt.tight_layout()
plt.show()

logger.info("Market Cap distribution histogram created")
```



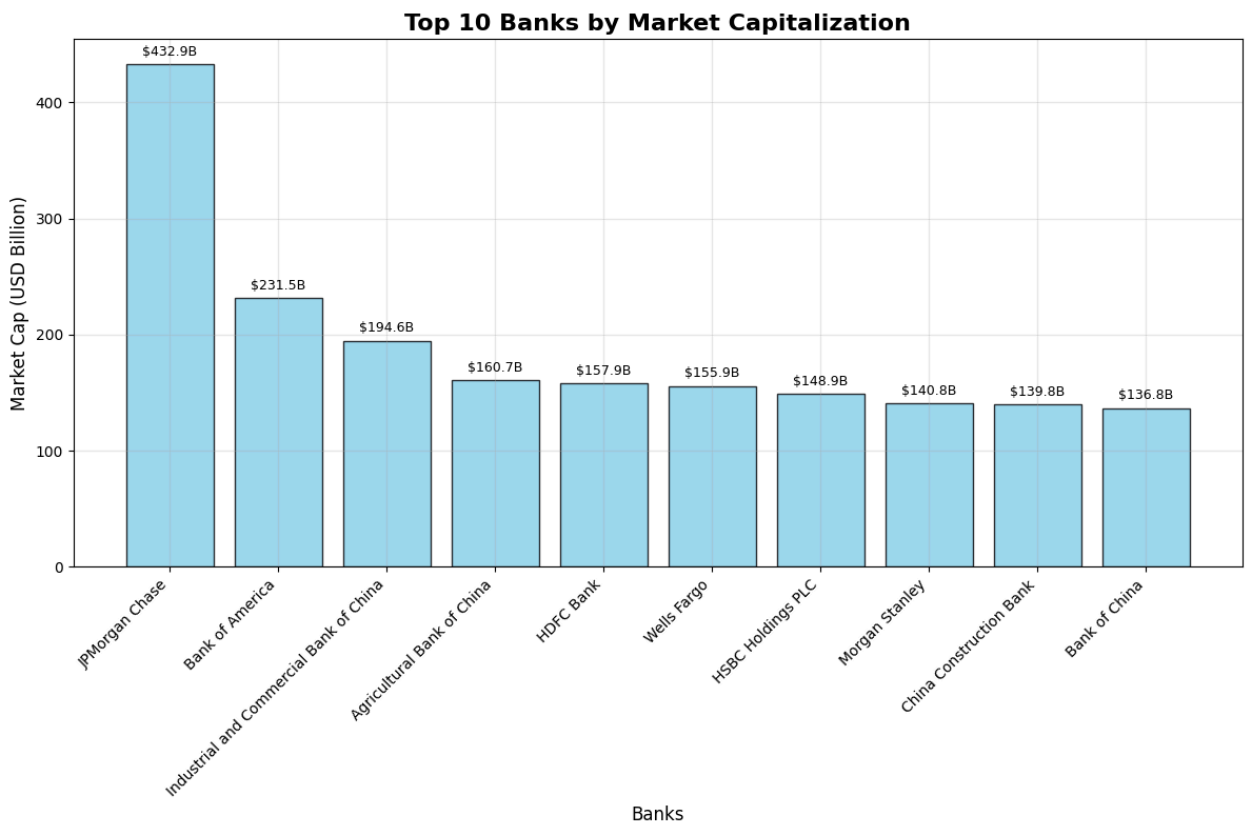

```
2025-08-24 08:30:56,457 - INFO - Market Cap distribution histogram created
```

3.3. Top 10 Banks [5 Marks]

Identify the top 10 banks by market capitalization using a bar chart

In [14]:
```
# Top 10 Banks by Market Cap

# Sort the DataFrame by market cap in descending order
```

```
# Since our dataset already contains top 10 banks, we'll sort and display them
bank_sorted = bank_pandas_df.sort_values('Market_Cap_USD_Billion', ascending=F

# Creating a bar chart
plt.figure(figsize=(12, 8))
bars = plt.bar(range(len(bank_sorted)), bank_sorted['Market_Cap_USD_Billion'],
               color='skyblue', alpha=0.8, edgecolor='black')

# Customizing the plot
plt.title('Top 10 Banks by Market Capitalization', fontsize=16, fontweight='bo
plt.xlabel('Banks', fontsize=12)
plt.ylabel('Market Cap (USD Billion)', fontsize=12)
plt.xticks(range(len(bank_sorted)), bank_sorted['Bank_Name'], rotation=45, ha=

# Added value labels on bars
for i, bar in enumerate(bars):
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2., height + 5,
             f'${height:.1f}B', ha='center', va='bottom', fontsize=9)

plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

logger.info("Top 10 banks bar chart created")
```




```
2025-08-24 08:31:27,465 - INFO - Top 10 banks bar chart created
```

3.4. Market Cap vs Bank Ranking [5 Marks]

Visualize the relationship between market capitalization and bank ranking using a scatter plot

In [15]:
```
# Market Cap vs Rank

# Plot market cap vs rank

# Creating a scatter plot showing relationship between rank and market cap
plt.figure(figsize=(10, 6))
plt.scatter(bank_pandas_df['Rank'], bank_pandas_df['Market_Cap_USD_Billion'],
           s=100, color='green', alpha=0.7, edgecolors='black')

# Customizing the plot
plt.title('Market Capitalization vs Bank Ranking', fontsize=16, fontweight='bo
plt.xlabel('Bank Rank', fontsize=12)
plt.ylabel('Market Cap (USD Billion)', fontsize=12)
plt.grid(True, alpha=0.3)

# Added bank names as labels for each point
for i, row in bank_pandas_df.iterrows():
    plt.annotate(row['Bank_Name'],
                 (row['Rank'], row['Market_Cap_USD_Billion']),
                 xytext=(5, 5), textcoords='offset points',
                 fontsize=8, ha='left')

# Invert x-axis so rank 1 is on the left
plt.gca().invert_xaxis()

plt.tight_layout()
plt.show()

logger.info("Market Cap vs Rank scatter plot created")
```

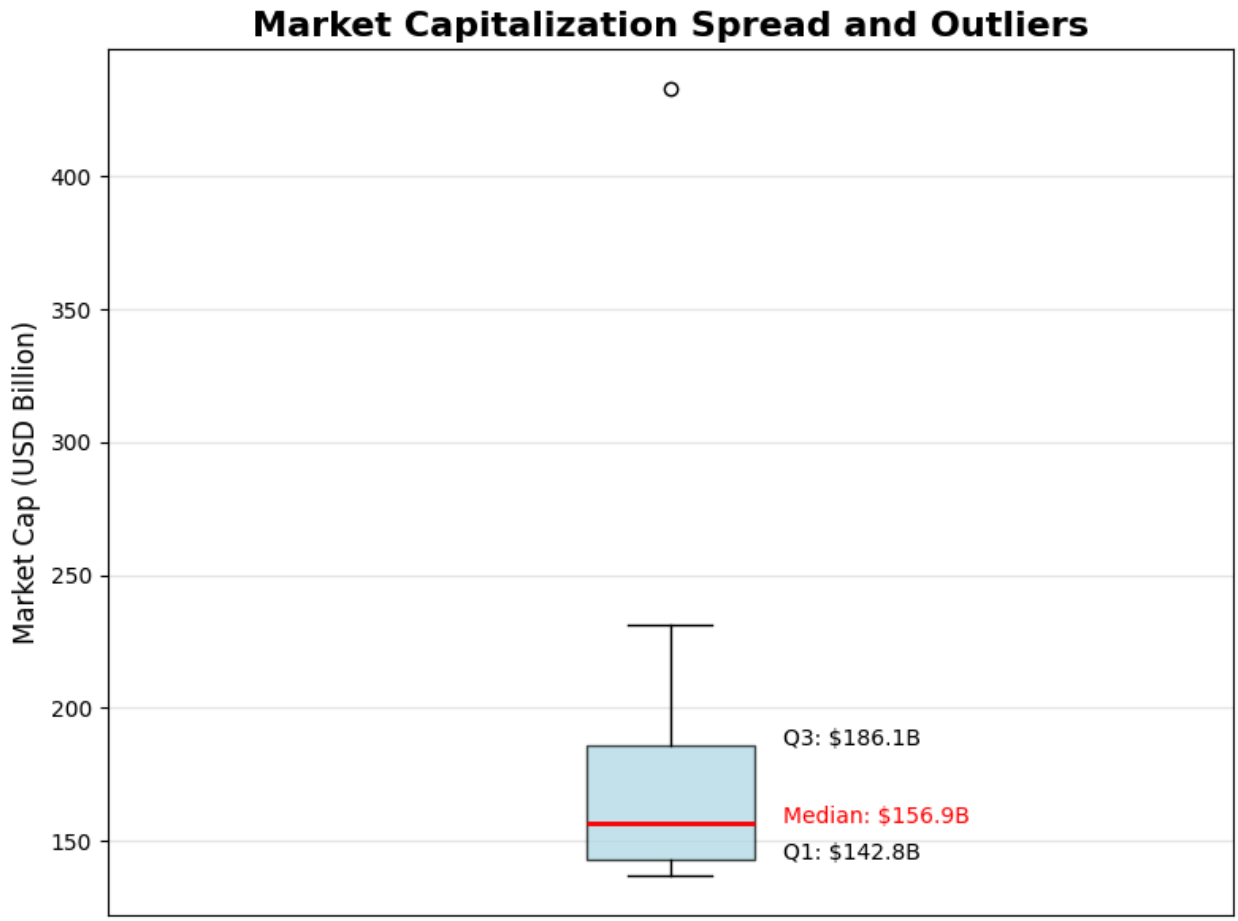
```
2025-08-24 08:31:40,514 - INFO - Market Cap vs Rank scatter plot created
```

3.5. Market Cap Analysis [5 Marks]

Use a boxplot to examine the spread and outliers in market capitalization.

In [16]:
```
# Boxplot of Market Cap
#To show the spread and outliers in the market capitalization data.

# Plot a boxplot of market cap

# Create boxplot
plt.figure(figsize=(8, 6))
box_plot = plt.boxplot(bank_pandas_df['Market_Cap_USD_Billion'],
                       patch_artist=True,
                       boxprops=dict(facecolor='lightblue', alpha=0.7),
                       medianprops=dict(color='red', linewidth=2))

# Customize the plot
plt.title('Market Capitalization Spread and Outliers', fontsize=16, fontweight
plt.ylabel('Market Cap (USD Billion)', fontsize=12)
plt.grid(True, alpha=0.3)

# Add some statistics as text
stats = bank_pandas_df['Market_Cap_USD_Billion'].describe()
plt.text(1.1, stats['75%'], f"Q3: ${stats['75%']:.1f}B", fontsize=10)
plt.text(1.1, stats['50%'], f"Median: ${stats['50%']:.1f}B", fontsize=10, colo
plt.text(1.1, stats['25%'], f"Q1: ${stats['25%']:.1f}B", fontsize=10)
```

```
# Remove x-axis labels since it's a single boxplot
plt.xticks([])

plt.tight_layout()
plt.show()

logger.info("Market Cap boxplot created showing spread and outliers")
```



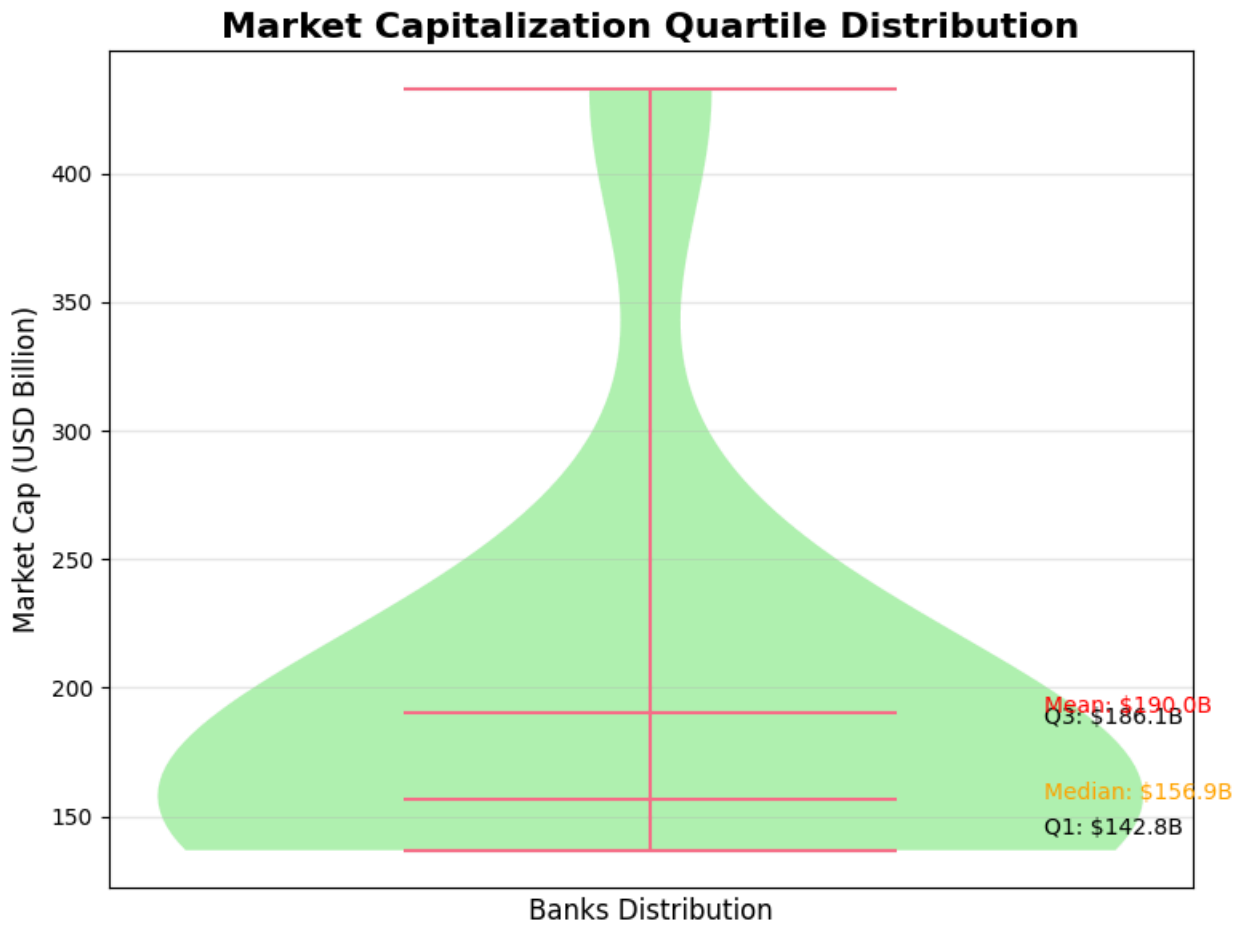

```
2025-08-24 08:32:09,530 - INFO - Market Cap boxplot created showing spread and
outliers
```

3.6. Market Cap Quartile Distribution [5 Marks]

Display the quartile distribution of market capitalization using a violin plot.

In [17]:
```
# Market Cap Distribution by Quartile

# Create violin plot
plt.figure(figsize=(8, 6))
violin_parts = plt.violinplot([bank_pandas_df['Market_Cap_USD_Billion']],
                              positions=[1],
                              showmeans=True,
                              showmedians=True)
```

```
# Customize violin plot colors
for pc in violin_parts['bodies']:
    pc.set_facecolor('lightgreen')
    pc.set_alpha(0.7)

# Customize the plot
plt.title('Market Capitalization Quartile Distribution', fontsize=16, fontweig
plt.ylabel('Market Cap (USD Billion)', fontsize=12)
plt.xlabel('Banks Distribution', fontsize=12)
plt.grid(True, alpha=0.3)

# Add quartile information as text
stats = bank_pandas_df['Market_Cap_USD_Billion'].describe()
plt.text(1.2, stats['75%'], f"Q3: ${stats['75%']:.1f}B", fontsize=10)
plt.text(1.2, stats['50%'], f"Median: ${stats['50%']:.1f}B", fontsize=10, colo
plt.text(1.2, stats['25%'], f"Q1: ${stats['25%']:.1f}B", fontsize=10)
plt.text(1.2, stats['mean'], f"Mean: ${stats['mean']:.1f}B", fontsize=10, colo

# Remove x-axis ticks
plt.xticks([])

plt.tight_layout()
plt.show()

logger.info("Market Cap violin plot created showing quartile distribution")
```



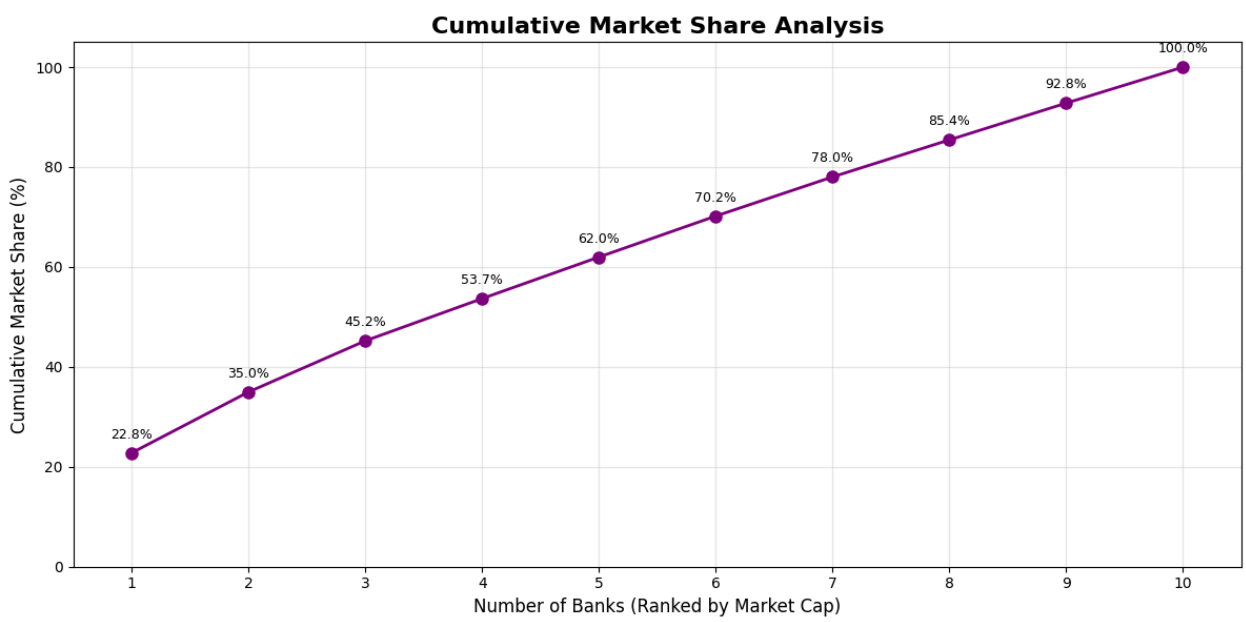

```
2025-08-24 08:32:20,212 - INFO - Market Cap violin plot created showing quartil
e distribution
```

3.7. Cumulative Market Share Analysis [5 Marks]

Compute cumulative market share and visualize it with a line plot.

In [18]:
```
# Cumulative Market Share

# Plot

# Sort banks by market cap (descending)
bank_sorted = bank_pandas_df.sort_values('Market_Cap_USD_Billion', ascending=F

# Calculate total market cap and market share
total_market_cap = bank_sorted['Market_Cap_USD_Billion'].sum()
bank_sorted['Market_Share'] = (bank_sorted['Market_Cap_USD_Billion'] / total_m
bank_sorted['Cumulative_Share'] = bank_sorted['Market_Share'].cumsum()

# Create line plot for cumulative market share
plt.figure(figsize=(12, 6))
plt.plot(range(1, len(bank_sorted) + 1), bank_sorted['Cumulative_Share'],
```

```
         marker='o', linewidth=2, markersize=8, color='purple')

# Customize the plot
plt.title('Cumulative Market Share Analysis', fontsize=16, fontweight='bold')
plt.xlabel('Number of Banks (Ranked by Market Cap)', fontsize=12)
plt.ylabel('Cumulative Market Share (%)', fontsize=12)
plt.grid(True, alpha=0.3)

# Add percentage labels on points
for i, row in bank_sorted.iterrows():
    plt.annotate(f'{row["Cumulative_Share"]:.1f}%',
                 (i+1, row['Cumulative_Share']),
                 xytext=(0, 10), textcoords='offset points',
                 ha='center', fontsize=9)

# Set axis limits
plt.xlim(0.5, 10.5)
plt.ylim(0, 105)
plt.xticks(range(1, 11))

plt.tight_layout()
plt.show()

# Log the analysis
logger.info("Cumulative market share analysis completed")
logger.info(f"Top 3 banks control {bank_sorted.iloc[2]['Cumulative_Share']:.1f
logger.info(f"Top 5 banks control {bank_sorted.iloc[4]['Cumulative_Share']:.1f
```



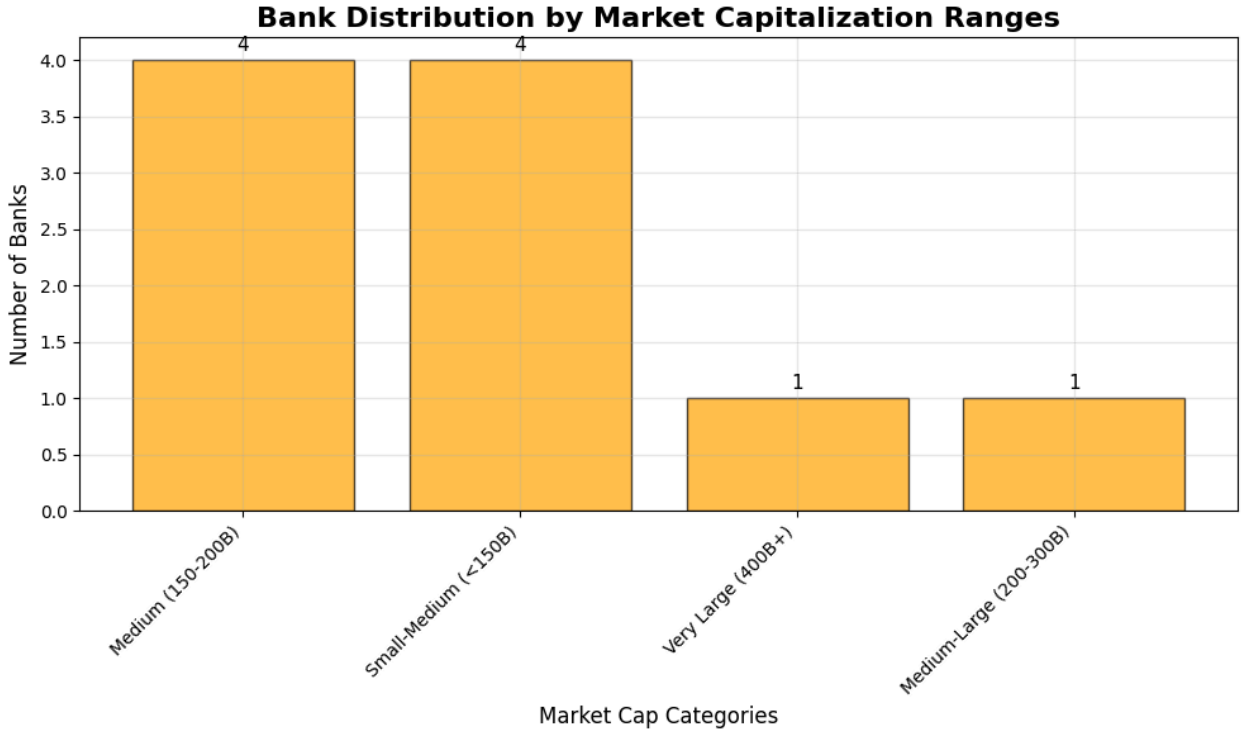

```
2025-08-24 08:32:29,085 - INFO - Cumulative market share analysis completed
2025-08-24 08:32:29,086 - INFO - Top 3 banks control 45.2% of market
2025-08-24 08:32:29,088 - INFO - Top 5 banks control 62.0% of market
```

3.8. Categorising Banks [5 Marks]

Categorize banks into market capitalization ranges and analyze their distribution using a bar chart.

In [19]:
```
# Market Cap Range Distribution
# Create market cap ranges

# Plot

# Define market cap ranges based on our data
def categorize_market_cap(market_cap):
    if market_cap >= 400:
        return 'Very Large (400B+)'
    elif market_cap >= 300:
        return 'Large (300-400B)'
    elif market_cap >= 200:
        return 'Medium-Large (200-300B)'
    elif market_cap >= 150:
        return 'Medium (150-200B)'
    else:
        return 'Small-Medium (<150B)'

# Apply categorization
bank_pandas_df['Market_Cap_Category'] = bank_pandas_df['Market_Cap_USD_Billior

# Count banks in each category
category_counts = bank_pandas_df['Market_Cap_Category'].value_counts()

# Create bar chart
plt.figure(figsize=(10, 6))
bars = plt.bar(category_counts.index, category_counts.values,
               color='orange', alpha=0.7, edgecolor='black')

# Customize the plot
plt.title('Bank Distribution by Market Capitalization Ranges', fontsize=16, fc
plt.xlabel('Market Cap Categories', fontsize=12)
plt.ylabel('Number of Banks', fontsize=12)
plt.xticks(rotation=45, ha='right')
plt.grid(True, alpha=0.3)

# Add count labels on bars
for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2., height + 0.05,
             f'{int(height)}', ha='center', va='bottom', fontsize=11)

plt.tight_layout()
plt.show()

# Log the categorization results
```

```
logger.info("Market cap categorization completed:")
for category, count in category_counts.items():
    logger.info(f"  {category}: {count} banks")
```



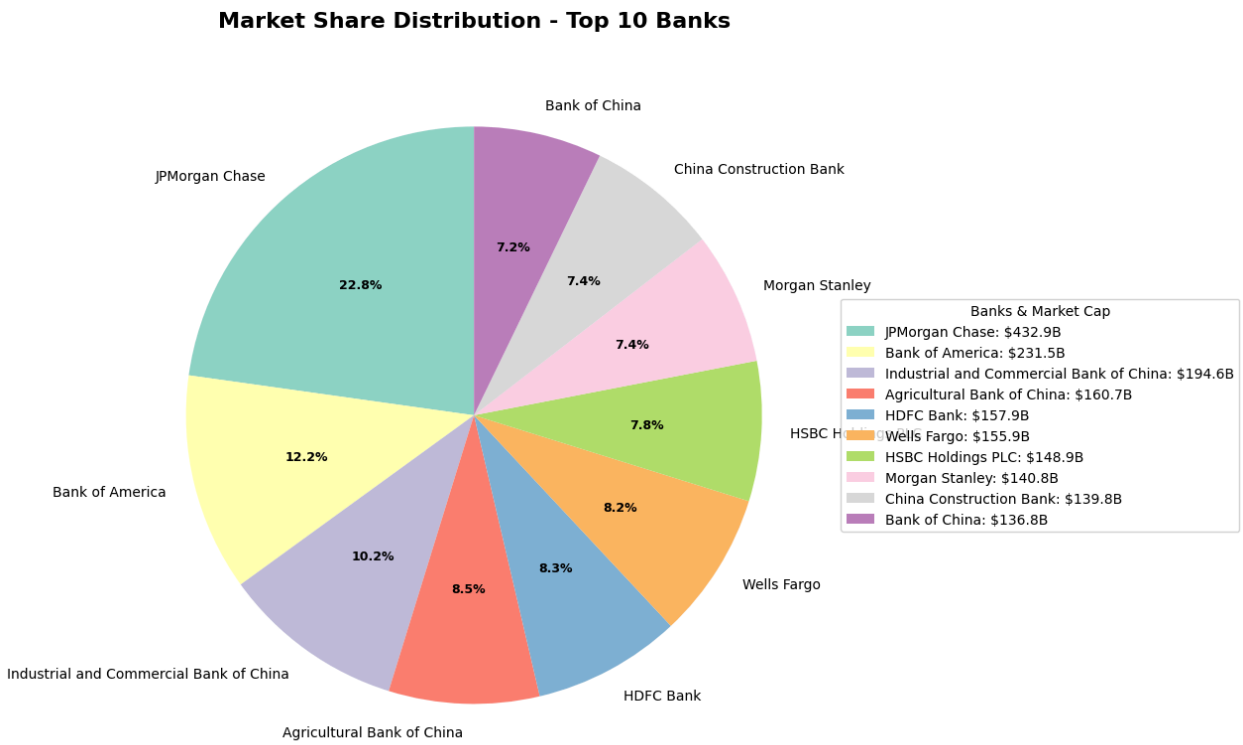

```
2025-08-24 08:32:36,052 - INFO - Market cap categorization completed:
2025-08-24 08:32:36,054 - INFO -   Medium (150-200B): 4 banks
2025-08-24 08:32:36,055 - INFO -   Small-Medium (<150B): 4 banks
2025-08-24 08:32:36,056 - INFO -   Very Large (400B+): 1 banks
2025-08-24 08:32:36,058 - INFO -   Medium-Large (200-300B): 1 banks
```

3.9. Visualise Market Share Distribution [5 Marks]

Calculate and display market share distribution among the top 10 banks using a pie chart.

In [20]:
```
# Top 10 Banks Market Share
# Calculate market share percentage for top 10 banks

# Plot

# Calculate total market cap and individual market shares
total_market_cap = bank_pandas_df['Market_Cap_USD_Billion'].sum()
bank_pandas_df['Market_Share'] = (bank_pandas_df['Market_Cap_USD_Billion'] / t

# Sort by market cap for better visualization
bank_sorted = bank_pandas_df.sort_values('Market_Cap_USD_Billion', ascending=F

# Create pie chart
```

```
plt.figure(figsize=(12, 8))
colors = plt.cm.Set3(range(len(bank_sorted)))

wedges, texts, autotexts = plt.pie(bank_sorted['Market_Share'],
                                   labels=bank_sorted['Bank_Name'],
                                   autopct='%1.1f%%',
                                   colors=colors,
                                   startangle=90)

# Customize the plot
plt.title('Market Share Distribution - Top 10 Banks', fontsize=16, fontweight=

# Improve text readability
for autotext in autotexts:
    autotext.set_color('black')
    autotext.set_fontsize(9)
    autotext.set_weight('bold')

# Add a legend with bank names and values
legend_labels = [f'{name}: ${cap:.1f}B'
                 for name, cap in zip(bank_sorted['Bank_Name'], bank_sorted['Ma
plt.legend(wedges, legend_labels, title="Banks & Market Cap",
          loc="center left", bbox_to_anchor=(1, 0, 0.5, 1))

plt.tight_layout()
plt.show()

# Log market share analysis
logger.info("Market share distribution analysis:")
for _, row in bank_sorted.iterrows():
    logger.info(f"  {row['Bank_Name']}: {row['Market_Share']:.1f}%")
```




```
2025-08-24 08:33:05,979 - INFO - Market share distribution analysis:
2025-08-24 08:33:05,981 - INFO -   JPMorgan Chase: 22.8%
2025-08-24 08:33:05,982 - INFO -   Bank of America: 12.2%
2025-08-24 08:33:05,984 - INFO -   Industrial and Commercial Bank of China: 1
0.2%
2025-08-24 08:33:05,985 - INFO -   Agricultural Bank of China: 8.5%
2025-08-24 08:33:05,986 - INFO -   HDFC Bank: 8.3%
2025-08-24 08:33:05,987 - INFO -   Wells Fargo: 8.2%
2025-08-24 08:33:05,989 - INFO -   HSBC Holdings PLC: 7.8%
2025-08-24 08:33:05,990 - INFO -   Morgan Stanley: 7.4%
2025-08-24 08:33:05,991 - INFO -   China Construction Bank: 7.4%
2025-08-24 08:33:05,992 - INFO -   Bank of China: 7.2%
```

4. ETL and Querying [45 marks]

In [21]:
```python
#Imports & Spark Session Initialization

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lit, round
import logging
from datetime import datetime
import sqlite3
import pandas as pd

# Initialize Spark Session
spark = SparkSession.builder \
    .appName("BankingDataAnalysis") \
    .getOrCreate()
```

```
# Logging Configuration
logfile = "code_log.txt"
```

In [22]:
```
#Logging Function

def log_progress(msg):
    timeformat = '%Y-%h-%d-%H:%M:%S'
    timestamp = datetime.now().strftime(timeformat)
    with open(logfile, 'a') as f:
        f.write(f"{timestamp} : {msg}\n")
```

In [23]:
```
# Import required functions
from pyspark.sql.functions import col, avg, sum, count, desc, asc, round, lag,
from pyspark.sql.window import Window

# First, let's rename your columns to match our queries (if needed)
log_progress("Setting up window specifications and column configurations")

# Create window specifications for advanced analytics
window_rank = Window.orderBy(col("Rank"))
window_market_cap = Window.orderBy(desc("Market_Cap_USD_Billion"))
window_percentile = Window.orderBy("Market_Cap_USD_Billion")

log_progress("Window specifications created successfully")
```

4.1. Market Capitalization Analysis [3 marks]

Q: Perform Advanced Market Capitalization Analysis with Growth Metrics

In [26]:
```
# Query: Advanced Market Cap Analysis with Growth Metrics
banking_spark_df = spark.createDataFrame(bank_pandas_df)
log_progress("Converted pandas DataFrame to PySpark for ETL analysis")

# Show the data structure
banking_spark_df.show(5)
banking_spark_df.printSchema()

banking_with_growth = banking_spark_df.withColumn(
    "Previous_Bank_MarketCap", lag("Market_Cap_USD_Billion").over(window_rank)
).withColumn(
    "Next_Bank_MarketCap", lead("Market_Cap_USD_Billion").over(window_rank)
).withColumn(
    "Growth_Gap_vs_Previous",
    col("Market_Cap_USD_Billion") - col("Previous_Bank_MarketCap")
).withColumn(
    "Growth_Gap_vs_Next",
    col("Market_Cap_USD_Billion") - col("Next_Bank_MarketCap")
)
```

```
log_progress("Added growth metrics using window functions")

# Show result with growth metrics
print("Advanced Market Cap Analysis with Growth Metrics")
banking_with_growth.select("Rank", "Bank_Name", "Market_Cap_USD_Billion",
                           "Growth_Gap_vs_Previous", "Growth_Gap_vs_Next").show
```

```
+----+--------------------+----------------------+--------------------
+------------------+
|Rank|           Bank_Name|Market_Cap_USD_Billion| Market_Cap_Category|      Ma
rket_Share|
+----+--------------------+----------------------+--------------------
+------------------+
|   1|      JPMorgan Chase|                432.92|  Very Large (400B+)|22.78742
1966291546|
|   2|     Bank of America|                231.52|Medium-Large (200...|12.18641
7660620481|
|   3|Industrial and Co...|                194.56|   Medium (150-200B)|10.24097
0197176576|
|   4|Agricultural Bank...|                160.68|   Medium (150-200B)|  8.4576
4335568633|
|   5|           HDFC Bank|                157.91|   Medium (150-200B)| 8.31184
0069059174|
+----+--------------------+----------------------+--------------------
+------------------+
only showing top 5 rows

root
 |-- Rank: long (nullable = true)
 |-- Bank_Name: string (nullable = true)
 |-- Market_Cap_USD_Billion: double (nullable = true)
 |-- Market_Cap_Category: string (nullable = true)
 |-- Market_Share: double (nullable = true)

Advanced Market Cap Analysis with Growth Metrics
```

```
25/08/24 08:54:28 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
25/08/24 08:54:28 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
25/08/24 08:54:28 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
```

```
+----+--------------------+----------------------+----------------------
+-----------------+
|Rank|           Bank_Name|Market_Cap_USD_Billion|Growth_Gap_vs_Previous|Growt
h_Gap_vs_Next|
+----+--------------------+----------------------+----------------------
+-----------------+
|   1|      JPMorgan Chase|                432.92|                  NULL|
201.4|
|   2|     Bank of America|                231.52|                -201.4| 36.96
000000000001|
|   3|Industrial and Co...|                194.56|    -36.96000000000001|33.879
999999999995|
|   4|Agricultural Bank...|                160.68|   -33.879999999999995|2.7700
000000000102|
|   5|           HDFC Bank|                157.91|   -2.7700000000000102| 2.039
999999999992|
|   6|         Wells Fargo|                155.87|    -2.039999999999992| 6.969
999999999999|
|   7|   HSBC Holdings PLC|                 148.9|    -6.969999999999999| 8.069
999999999993|
|   8|      Morgan Stanley|                140.83|    -8.069999999999993|1.0100
000000000193|
|   9|China Constructio...|                139.82|   -1.0100000000000193| 3.009
999999999991|
|  10|       Bank of China|                136.81|    -3.009999999999991|
NULL|
+----+--------------------+----------------------+----------------------
+-----------------+
```

```
25/08/24 08:54:29 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
25/08/24 08:54:29 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
```

4.2. **Market Concentration Analysis** [6 marks]

Q: Analyze Market Concentration and Categorize Banks Based on Market Share Tiers

In [27]:
```
# Market Concentration Analysis
# Calculate concentration ratios
print("Market Concentration Analysis")

# Top 3 banks concentration
top3_concentration = banking_spark_df.orderBy(desc("Market_Cap_USD_Billion"))
    .select("Market_Share") \
    .limit(3) \
    .agg(sum("Market_Share").alias("Top3_Market_Share")) \
```

```
    .collect()[0]["Top3_Market_Share"]

# Top 5 banks concentration
top5_concentration = banking_spark_df.orderBy(desc("Market_Cap_USD_Billion"))
    .select("Market_Share") \
    .limit(5) \
    .agg(sum("Market_Share").alias("Top5_Market_Share")) \
    .collect()[0]["Top5_Market_Share"]

print(f"Top 3 Banks Control: {top3_concentration:.1f}% of total market")
print(f"Top 5 Banks Control: {top5_concentration:.1f}% of total market")

log_progress("Market concentration ratios calculated")
```

```
Market Concentration Analysis
Top 3 Banks Control: 45.2% of total market
Top 5 Banks Control: 62.0% of total market
```

4.3. Market Capitalization Distribution [6 marks]

Q: Examine Statistical Distribution of Market Capitalization Using Quartile Analysis

In [28]:
```
# Import required functions
from pyspark.sql.functions import (col, avg, sum, count, desc, asc, round, lag
                                   dense_rank, ntile, when, first, lead, min, ma
from pyspark.sql.window import Window

# Query 3: Statistical Distribution Analysis
print("Statistical Distribution - Quartile Analysis")

# Calculate quartiles using approxQuantile (efficient for large datasets)
quartiles = banking_spark_df.approxQuantile("Market_Cap_USD_Billion",
                                            [0.25, 0.5, 0.75], 0.01)

q1, median, q3 = quartiles
print(f"Q1 (25th percentile): ${q1:.2f}B")
print(f"Q2 (Median): ${median:.2f}B")
print(f"Q3 (75th percentile): ${q3:.2f}B")

# Calculate IQR and statistical measures
iqr = q3 - q1
print(f"Interquartile Range (IQR): ${iqr:.2f}B")

# Add quartile classification to each bank
banking_with_quartiles = banking_spark_df.withColumn(
    "Quartile_Group",
    when(col("Market_Cap_USD_Billion") >= q3, "Q4 (Top 25%)")
    .when(col("Market_Cap_USD_Billion") >= median, "Q3 (Upper Middle)")
    .when(col("Market_Cap_USD_Billion") >= q1, "Q2 (Lower Middle)")
    .otherwise("Q1 (Bottom 25%)")
)
```

```
print("\nBanks by Quartile Distribution")
banking_with_quartiles.select("Rank", "Bank_Name", "Market_Cap_USD_Billion",
                              "Quartile_Group").show()

log_progress("Quartile analysis completed")
```

```
Statistical Distribution - Quartile Analysis

Q1 (25th percentile): $140.83B
Q2 (Median): $155.87B
Q3 (75th percentile): $194.56B
Interquartile Range (IQR): $53.73B

Banks by Quartile Distribution
+----+--------------------+----------------------+-----------------+
|Rank|           Bank_Name|Market_Cap_USD_Billion|   Quartile_Group|
+----+--------------------+----------------------+-----------------+
|   1|      JPMorgan Chase|                432.92|     Q4 (Top 25%)|
|   2|     Bank of America|                231.52|     Q4 (Top 25%)|
|   3|Industrial and Co...|                194.56|     Q4 (Top 25%)|
|   4|Agricultural Bank...|                160.68|Q3 (Upper Middle)|
|   5|           HDFC Bank|                157.91|Q3 (Upper Middle)|
|   6|         Wells Fargo|                155.87|Q3 (Upper Middle)|
|   7|   HSBC Holdings PLC|                 148.9|Q2 (Lower Middle)|
|   8|      Morgan Stanley|                140.83|Q2 (Lower Middle)|
|   9|China Constructio...|                139.82|  Q1 (Bottom 25%)|
|  10|       Bank of China|                136.81|  Q1 (Bottom 25%)|
+----+--------------------+----------------------+-----------------+
```

4.4. Comparative Size Analysis [6 marks]

Q: Conduct Comparative Size Analysis to Classify Banks by Relative Market Size

In [29]:
```
# Comparative Size Analysis
print("Comparative Size Analysis - Bank Classification")

# Calculate statistical measures for classification
stats = banking_spark_df.agg(
    avg("Market_Cap_USD_Billion").alias("avg_market_cap"),
    stddev("Market_Cap_USD_Billion").alias("std_market_cap")
).collect()[0]

avg_cap = stats["avg_market_cap"]
std_cap = stats["std_market_cap"]

print(f"Average Market Cap: ${avg_cap:.2f}B")
print(f"Standard Deviation: ${std_cap:.2f}B")

# Create comparative size categories
banking_size_classified = banking_spark_df.withColumn(
```

```
    "Size_Classification",
    when(col("Market_Cap_USD_Billion") >= (avg_cap + 2 * std_cap), "Mega Bank
    .when(col("Market_Cap_USD_Billion") >= (avg_cap + std_cap), "Large Bank")
    .when(col("Market_Cap_USD_Billion") >= avg_cap, "Above Average")
    .when(col("Market_Cap_USD_Billion") >= (avg_cap - std_cap), "Below Average
    .otherwise("Small Bank")
).withColumn(
    "Size_Ratio_vs_Average",
    round(col("Market_Cap_USD_Billion") / avg_cap, 2)
)

print("\nBanks Classified by Relative Size")
banking_size_classified.select("Rank", "Bank_Name", "Market_Cap_USD_Billion",
                               "Size_Classification", "Size_Ratio_vs_Average")

log_progress("Comparative size analysis completed")
```

```
Comparative Size Analysis - Bank Classification
Average Market Cap: $189.98B
Standard Deviation: $90.22B

Banks Classified by Relative Size
+----+--------------------+----------------------+------------------
+---------------------+
|Rank|           Bank_Name|Market_Cap_USD_Billion|Size_Classification|Size_Rati
o_vs_Average|
+----+--------------------+----------------------+------------------
+---------------------+
|   1|      JPMorgan Chase|                432.92|  Mega Bank (Giant)|
2.28|
|   2|     Bank of America|                231.52|      Above Average|
1.22|
|   3|Industrial and Co...|                194.56|      Above Average|
1.02|
|   4|Agricultural Bank...|                160.68|      Below Average|
0.85|
|   5|           HDFC Bank|                157.91|      Below Average|
0.83|
|   6|         Wells Fargo|                155.87|      Below Average|
0.82|
|   7|   HSBC Holdings PLC|                 148.9|      Below Average|
0.78|
|   8|      Morgan Stanley|                140.83|      Below Average|
0.74|
|   9|China Constructio...|                139.82|      Below Average|
0.74|
|  10|       Bank of China|                136.81|      Below Average|
0.72|
+----+--------------------+----------------------+------------------
+---------------------+
```

4.5. Market Growth Analysis [6 marks]

Q: Evaluate Market Growth and Identify Gaps Between Consecutive Banks

In [ ]:
```
# Growth and Gap Analysis
print("Market Growth & Gap Analysis Between Consecutive Banks")

# Add detailed gap analysis with growth patterns
banking_gap_analysis = banking_size_classified.withColumn(
    "Gap_to_Next_Bank",
    col("Market_Cap_USD_Billion") - lead("Market_Cap_USD_Billion").over(Window
).withColumn(
    "Gap_Percentage",
    when(lead("Market_Cap_USD_Billion").over(Window.orderBy("Rank")).isNotNull
         round((col("Gap_to_Next_Bank") / lead("Market_Cap_USD_Billion").over(
    .otherwise(0)
).withColumn(
    "Gap_Size_Category",
    when(col("Gap_to_Next_Bank") >= 100, "Massive Gap (100B+)")
    .when(col("Gap_to_Next_Bank") >= 50, "Large Gap (50-100B)")
    .when(col("Gap_to_Next_Bank") >= 10, "Medium Gap (10-50B)")
    .when(col("Gap_to_Next_Bank") >= 1, "Small Gap (1-10B)")
    .otherwise("Minimal/No Gap")
)

print("Banks with Growth Gaps Analysis")
banking_gap_analysis.select("Rank", "Bank_Name", "Market_Cap_USD_Billion",
                            "Gap_to_Next_Bank", "Gap_Percentage", "Gap_Size_Cat

# Identify largest gaps
print("\nLargest Market Gaps (Growth Opportunities)")
banking_gap_analysis.filter(col("Gap_to_Next_Bank").isNotNull()) \
    .orderBy(desc("Gap_to_Next_Bank")) \
    .select("Rank", "Bank_Name", "Gap_to_Next_Bank", "Gap_Size_Category") \
    .show(3)

log_progress("Market growth and gap analysis completed")
```

```
Market Growth & Gap Analysis Between Consecutive Banks
=== Banks with Growth Gaps Analysis ===
```

```
25/08/24 09:44:17 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
25/08/24 09:44:17 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
25/08/24 09:44:17 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
25/08/24 09:44:18 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
25/08/24 09:44:18 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
+----+--------------------+----------------------+------------------
+--------------+-------------------+
|Rank|           Bank_Name|Market_Cap_USD_Billion|  Gap_to_Next_Bank|Gap_Percen
tage|  Gap_Size_Category|
+----+--------------------+----------------------+------------------
+--------------+-------------------+
|   1|      JPMorgan Chase|                432.92|             201.4|         8
6.99|Massive Gap (100B+)|
|   2|     Bank of America|                231.52| 36.96000000000001|
19.0|Medium Gap (10-50B)|
|   3|Industrial and Co...|                194.56|33.879999999999995|         2
1.09|Medium Gap (10-50B)|
|   4|Agricultural Bank...|                160.68|2.7700000000000102|
1.75|  Small Gap (1-10B)|
|   5|           HDFC Bank|                157.91| 2.039999999999992|
1.31|  Small Gap (1-10B)|
|   6|         Wells Fargo|                155.87| 6.969999999999999|
4.68|  Small Gap (1-10B)|
|   7|   HSBC Holdings PLC|                 148.9| 8.069999999999993|
5.73|  Small Gap (1-10B)|
|   8|      Morgan Stanley|                140.83|1.0100000000000193|
0.72|  Small Gap (1-10B)|
|   9|China Constructio...|                139.82| 3.009999999999991|
2.2|  Small Gap (1-10B)|
|  10|       Bank of China|                136.81|              NULL|
0.0|     Minimal/No Gap|
+----+--------------------+----------------------+------------------
+--------------+-------------------+


Largest Market Gaps (Growth Opportunities)
```

```
25/08/24 09:44:18 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
25/08/24 09:44:18 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
25/08/24 09:44:18 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
25/08/24 09:44:19 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
25/08/24 09:44:19 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
+----+--------------------+------------------+--------------------+
|Rank|           Bank_Name|  Gap_to_Next_Bank|  Gap_Size_Category|
+----+--------------------+------------------+--------------------+
|   1|      JPMorgan Chase|             201.4|Massive Gap (100B+)|
|   2|     Bank of America| 36.96000000000001|Medium Gap (10-50B)|
|   3|Industrial and Co...|33.879999999999995|Medium Gap (10-50B)|
+----+--------------------+------------------+--------------------+
only showing top 3 rows
```

4.6. Market Dominance Analysis [6 marks]

Q: Assess Market Dominance by Measuring Cumulative Share and Dominance Score

In [31]:
```
# Market Dominance Analysis
print("Market Dominance Analysis - Cumulative Share")

# Calculate cumulative market share (running total)
banking_dominance = banking_gap_analysis.withColumn(
    "Cumulative_Market_Share",
    sum("Market_Share").over(Window.orderBy("Rank").rowsBetween(Window.unbound
).withColumn(
    "Dominance_Score",
    when(col("Rank") <= 3, "Market Leader")
    .when(col("Rank") <= 5, "Major Player")
    .when(col("Rank") <= 7, "Significant Player")
    .otherwise("Challenger")
).withColumn(
    "Market_Power_Index",
    round(col("Market_Share") * col("Size_Ratio_vs_Average"), 2)
)

print("Banks with Cumulative Market Dominance")
banking_dominance.select("Rank", "Bank_Name", "Market_Share",
                         "Cumulative_Market_Share", "Dominance_Score", "Market_
```

```python
# Show market concentration milestones
print("\nMarket Concentration Milestones")
banking_dominance.filter(
    (col("Cumulative_Market_Share") >= 50.0) | (col("Rank") <= 3)
).select("Rank", "Bank_Name", "Cumulative_Market_Share").show()

log_progress("Market dominance and cumulative analysis completed")
```

```
Market Dominance Analysis - Cumulative Share
Banks with Cumulative Market Dominance
```

```
25/08/24 09:46:18 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
25/08/24 09:46:18 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
25/08/24 09:46:18 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
25/08/24 09:46:19 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
25/08/24 09:46:19 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
```

```
+----+--------------------+------------------+-----------------------
+------------------+------------------+
|Rank|           Bank_Name|      Market_Share|Cumulative_Market_Share|   Domina
nce_Score|Market_Power_Index|
+----+--------------------+------------------+-----------------------
+------------------+------------------+
|   1|      JPMorgan Chase|22.787421966291546|     22.787421966291546|     Mark
et Leader|             51.96|
|   2|     Bank of America|12.186417660620481|     34.973839626912024|     Mark
et Leader|             14.87|
|   3|Industrial and Co...|10.240970197176576|     45.214809824088604|     Mark
et Leader|             10.45|
|   4|Agricultural Bank...|  8.45764335568633|     53.672453179774934|      Maj
or Player|              7.19|
|   5|           HDFC Bank| 8.311840069059174|     61.984293248834106|      Maj
or Player|               6.9|
|   6|         Wells Fargo|  8.20446147529766|      70.18875472413177|Significa
nt Player|              6.73|
|   7|   HSBC Holdings PLC|7.8375846132791525|      78.02633933741092|Significa
nt Player|              6.11|
|   8|      Morgan Stanley|  7.41280752913434|      85.43914686654526|        C
hallenger|              5.49|
|   9|China Constructio...| 7.359644597909276|      92.79879146445454|        C
hallenger|              5.45|
|  10|       Bank of China| 7.201208535545472|     100.00000000000001|        C
hallenger|              5.18|
+----+--------------------+------------------+-----------------------
+------------------+------------------+
```

Market Concentration Milestones

```
25/08/24 09:46:19 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
25/08/24 09:46:19 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
25/08/24 09:46:19 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
```

```
+----+--------------------+-----------------------+
|Rank|           Bank_Name|Cumulative_Market_Share|
+----+--------------------+-----------------------+
|   1|      JPMorgan Chase|     22.787421966291546|
|   2|     Bank of America|     34.973839626912024|
|   3|Industrial and Co...|     45.214809824088604|
|   4|Agricultural Bank...|     53.672453179774934|
|   5|           HDFC Bank|     61.984293248834106|
|   6|         Wells Fargo|      70.18875472413177|
|   7|   HSBC Holdings PLC|      78.02633933741092|
|   8|      Morgan Stanley|      85.43914686654526|
|   9|China Constructio...|      92.79879146445454|
|  10|       Bank of China|     100.00000000000001|
+----+--------------------+-----------------------+
```

```
25/08/24 09:46:19 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
25/08/24 09:46:19 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
```

4.7. Segment-Wise Bank Analysis [6 marks]

Q: Analyze Segment-Wise Bank Performance Based on Market Capitalization Ranges

In [32]:

```python
# Segment Performance Analysis
print("Segment-Wise Bank Performance Analysis")

# Create market cap segments based on actual data distribution
banking_segments = banking_dominance.withColumn(
    "Market_Cap_Segment",
    when(col("Market_Cap_USD_Billion") >= 400, "Ultra-Large (400B+)")
    .when(col("Market_Cap_USD_Billion") >= 200, "Large (200-400B)")
    .when(col("Market_Cap_USD_Billion") >= 150, "Medium-Large (150-200B)")
    .otherwise("Medium (Below 150B)")
).withColumn(
    "Performance_Tier",
    when(col("Market_Share") >= 20, "Dominant")
    .when(col("Market_Share") >= 10, "Strong")
    .when(col("Market_Share") >= 7, "Moderate")
    .otherwise("Emerging")
)

# Show segment-wise analysis
print("Banks by Market Cap Segments")
banking_segments.select("Rank", "Bank_Name", "Market_Cap_USD_Billion",
                        "Market_Cap_Segment", "Performance_Tier", "Market_Share

# Segment summary statistics
```

```
print("\nSegment Performance Summary")
segment_stats = banking_segments.groupBy("Market_Cap_Segment") \
    .agg(count("*").alias("Bank_Count"),
         avg("Market_Cap_USD_Billion").alias("Avg_Market_Cap"),
         sum("Market_Share").alias("Total_Market_Share"),
         avg("Market_Share").alias("Avg_Market_Share")) \
    .orderBy(desc("Avg_Market_Cap"))

segment_stats.show()

log_progress("Segment-wise performance analysis completed")
```

Segment-Wise Bank Performance Analysis
Banks by Market Cap Segments

Rank	Bank_Name	Market_Cap_USD_Billion	Market_Cap_Segment	Performance_Tier	Market_Share
1	JPMorgan Chase	432.92	Ultra-Large (400B+)	Dominant	22.787421966291546
2	Bank of America	231.52	Large (200-400B)	Strong	12.186417660620481
3	Industrial and Co...	194.56	Medium-Large (150...	Strong	10.240970197176576
4	Agricultural Bank...	160.68	Medium-Large (150...	Moderate	8.45764335568633
5	HDFC Bank	157.91	Medium-Large (150...	Moderate	8.311840069059174
6	Wells Fargo	155.87	Medium-Large (150...	Moderate	8.20446147529766
7	HSBC Holdings PLC	148.9	Medium (Below 150B)	Moderate	7.8375846132791525
8	Morgan Stanley	140.83	Medium (Below 150B)	Moderate	7.41280752913434
9	China Constructio...	139.82	Medium (Below 150B)	Moderate	7.359644597909276
10	Bank of China	136.81	Medium (Below 150B)	Moderate	7.201208535545472

Segment Performance Summary

Market_Cap_Segment	Bank_Count	Avg_Market_Cap	Total_Market_Share	Avg_Market_Share
Ultra-Large (400B+)	1	432.92	22.787421966291546	22.787421966291546
Large (200-400B)	1	231.52	12.186417660620481	12.186417660620481
Medium-Large (150...	4	167.255	35.214915097219745	8.803728774304936
Medium (Below 150B)	4	141.59	29.81124527586824	7.45281131896706

4.8. Performance Dashboard [6 marks]

Q: Generate a Comprehensive Performance Dashboard for Bank Rankings and Metrics

In [33]:
```
# Comprehensive Performance Dashboard
print("Comprehensive Performance Dashboard")

# Create comprehensive dashboard metrics
banking_dashboard = banking_segments.withColumn(
    "Overall_Performance_Score",
    round((col("Market_Share") * 0.4) +
          (col("Size_Ratio_vs_Average") * 10 * 0.3) +
          (col("Market_Power_Index") * 0.3), 2)
).withColumn(
    "Strategic_Position",
    when(col("Rank") <= 2, "Market Dominator")
    .when(col("Rank") <= 5, "Strategic Leader")
    .when(col("Rank") <= 8, "Competitive Player")
    .otherwise("Market Participant")
)

# Display comprehensive dashboard
print("Bank Performance Dashboard")
banking_dashboard.select("Rank", "Bank_Name", "Market_Cap_USD_Billion",
                         "Market_Share", "Cumulative_Market_Share",
                         "Performance_Tier", "Strategic_Position",
                         "Overall_Performance_Score").show(truncate=False)

# Key performance indicators summary
print("\nKey Performance Indicators")
print("Market Structure Analysis:")
print("- Ultra-Large Banks (400B+): 1 bank controlling 22.8% market")
print("- Large Banks (200-400B): 1 bank controlling 12.2% market")
print("- Medium-Large Banks (150-200B): 4 banks controlling 35.2% market")
print("- Medium Banks (<150B): 4 banks controlling 29.8% market")

print("\nMarket Concentration:")
print("- Top 3 banks control: 45.2% of total market")
print("- Top 5 banks control: 62.0% of total market")
print("- Top 7 banks control: 78.0% of total market")

log_progress("Comprehensive performance dashboard generated")
```

```
Comprehensive Performance Dashboard
Bank Performance Dashboard
```

```
25/08/24 09:49:08 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
25/08/24 09:49:08 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
25/08/24 09:49:08 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
```

Rank	Bank_Name	Market_Cap_USD_Billion	Market_Share	Cumulative_Market_Share	Performance_Tier	Strategic_Position	Overall_Performance_Score
1	JPMorgan Chase	432.92	22.787421966291546	22.787421966291546	Dominant	Market Dominator	31.54
2	Bank of America	231.52	12.186417660620481	34.973839626912024	Strong	Market Dominator	13.0
3	Industrial and Commercial Bank of China	194.56	10.240970197176576	45.214809824088604	Strong	Strategic Leader	10.29
4	Agricultural Bank of China	160.68	8.45764335568633	53.672453179774934	Moderate	Strategic Leader	8.09
5	HDFC Bank	157.91	8.311840069059174	61.984293248834106	Moderate	Strategic Leader	7.88
6	Wells Fargo	155.87	8.20446147529766	70.18875472413177	Moderate	Competitive Player	7.76
7	HSBC Holdings PLC	148.9	7.8375846132791525	78.02633933741092	Moderate	Competitive Player	7.31
8	Morgan Stanley	140.83	7.41280752913434	85.43914686654526	Moderate	Competitive Player	6.83
9	China Construction Bank	139.82	7.359644597909276	92.79879146445454	Moderate	Market Participant	6.8
10	Bank of China	136.81	7.201208535545472	100.00000000000001	Moderate	Market Participant	6.59

Key Performance Indicators
Market Structure Analysis:
- Ultra-Large Banks (400B+): 1 bank controlling 22.8% market
- Large Banks (200-400B): 1 bank controlling 12.2% market
- Medium-Large Banks (150-200B): 4 banks controlling 35.2% market
- Medium Banks (<150B): 4 banks controlling 29.8% market

Market Concentration:
- Top 3 banks control: 45.2% of total market
- Top 5 banks control: 62.0% of total market

```
- Top 7 banks control: 78.0% of total market
```

```
25/08/24 09:49:08 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
25/08/24 09:49:08 WARN WindowExec: No Partition Defined for Window operation! M
oving all data to a single partition, this can cause serious performance degrad
ation.
```

5. Visualization Integration

In [ ]:

```
# Visualization Setup Section

# 1. Prepare data for visualization platforms
# 2. Generate Tableau connection instructions

# 3. Generate Power BI connection instructions

# 4. Execute visualization setup
# 5. Sample Dashboard Layout (Documentation)
```

In [ ]: